



BEAUTY OF CLOUD 2.0 REPORT & ARCHITECTURE DIAGRAM TEMPLATE



University of Sri Jayewardenepura
IEEE Student Branch



IEEE
COMPUTER
SOCIETY

University of Sri Jayewardenepura
Students' Branches & Chapters

- Team Name: BigBug
- Scenario Chosen: Scenario 5 — AI Agent Platform for Business Automation
- Cloud Platform(s) Used: GCP (Google Cloud Platform) — primary platform. Anthropic Claude API used as an external LLM fallback only; all core infrastructure runs on GCP.

Team Members

#	Full Name	Role in Team	University	Faculty	Email	Contact Number
1	Sandaru Nethmina Samarasekara	Team Lead	UOM	EFac	Sandaruns2004@gmail.com	0767588712
2	Nadija Welihena Gamage	LLM Orchestration	UOM	EFac	nadijasmwg@gmail.com	0722254703
3	Sithum Fernando	Backend Engineer	UOM	EFac	sithumdf@gmail.com	0774348111
4	Ranuja Jayawardena	Cloud Architect	UOM	EFac	ranujatkj@gmail.com	0711723936

(2–4 members per team, as per registration rules)

1. Section 1 - Problem Understanding

Businesses today want AI that can actually do things — not just answer questions in a chat window. A growing number of companies need AI agents that can look up a customer's order history, check internal policy documents, trigger a refund, or send a follow-up email — all autonomously, across multiple steps, without a human supervising each one.

The real problem is not the AI itself. The problem is everything that needs to exist **around** the AI to make it safe and reliable enough to trust in a real business environment. Consider what a company actually needs to deploy even a simple "AI customer support agent":

- The agent needs to remember what the customer said last week — but storing and retrieving that context efficiently, without stuffing a company's entire conversation history into every LLM call, is a significant engineering problem on its own.
- The agent needs to look up order status from a live internal database — but how do you let an AI make real API calls without risking it calling the wrong endpoint, passing bad parameters, or triggering an irreversible action based on a hallucinated output?
- The business needs to know why the agent gave a wrong answer — not just that it did. Debugging an LLM-based system that has no trace of its decision-making is nearly impossible in production.
- The business needs to control costs LLM inference charges by the token, and an unconstrained agent running thousands of conversations a day can generate bills far beyond what was budgeted.
- Multiple businesses on the same platform must be completely isolated from each other one company's data, agent behavior, and usage spikes must never affect another's.

Who is affected: Small and medium-sized businesses are the most severely impacted. Large enterprises can afford to hire teams of ML engineers and cloud architects to build this infrastructure from scratch. Smaller businesses cannot they are forced to either use a basic chatbot with no real capability, pay for a proprietary black-box product they cannot understand or audit, or attempt to build it themselves and spend months on infrastructure before writing a single line of actual business logic.

End users the customers interacting with these agents are affected when agents give confidently wrong answers, take unintended actions (cancelling a subscription instead of pausing it), or provide no way to escalate to a human when something goes wrong.

Why existing solutions fall short: Raw LLM APIs like Gemini or GPT-4 provide the model only — no memory across sessions, no tool-calling infrastructure, no guardrails, and no multi-tenancy. Frameworks like LangChain are exactly that: frameworks. They require a significant engineering team to turn into a production system. Managed agent services from individual cloud providers are deeply tied to one ecosystem and offer limited ability to customise safety

and isolation behavior. None of these give a business a "configure and deploy" path to a safe, observable, cost-controlled AI agent.

Our specific focus: Rather than building a shallow solution across every dimension, our team focused deeply on three interconnected areas:

1. **Safe tool-calling with a 3-layer guardrails pipeline** — the mechanism that allows agents to take real actions (calling APIs, triggering workflows) without risking harmful, incorrect, or unintended executions. This is the most dangerous part of any AI agent system and the part most likely to cause real-world harm if done carelessly.
2. **Tenant-isolated memory using RAG (Retrieval-Augmented Generation)** — allowing agents to remember relevant past context without exploding token costs, and ensuring no business's data ever appears in another business's agent responses.
3. **Per-decision audit trails** — a structured record of every step an agent took to reach a decision, stored in a queryable format so a business can answer "why did the agent do that?" for any interaction in history.

We explicitly did not go deep on the UI/UX of the agent configuration dashboard, the platform's own billing and payment systems, or LLM fine-tuning — all real requirements for a full production system, but outside our chosen focus for this proposal.

2. Section 2 - Proposed Solution Overview

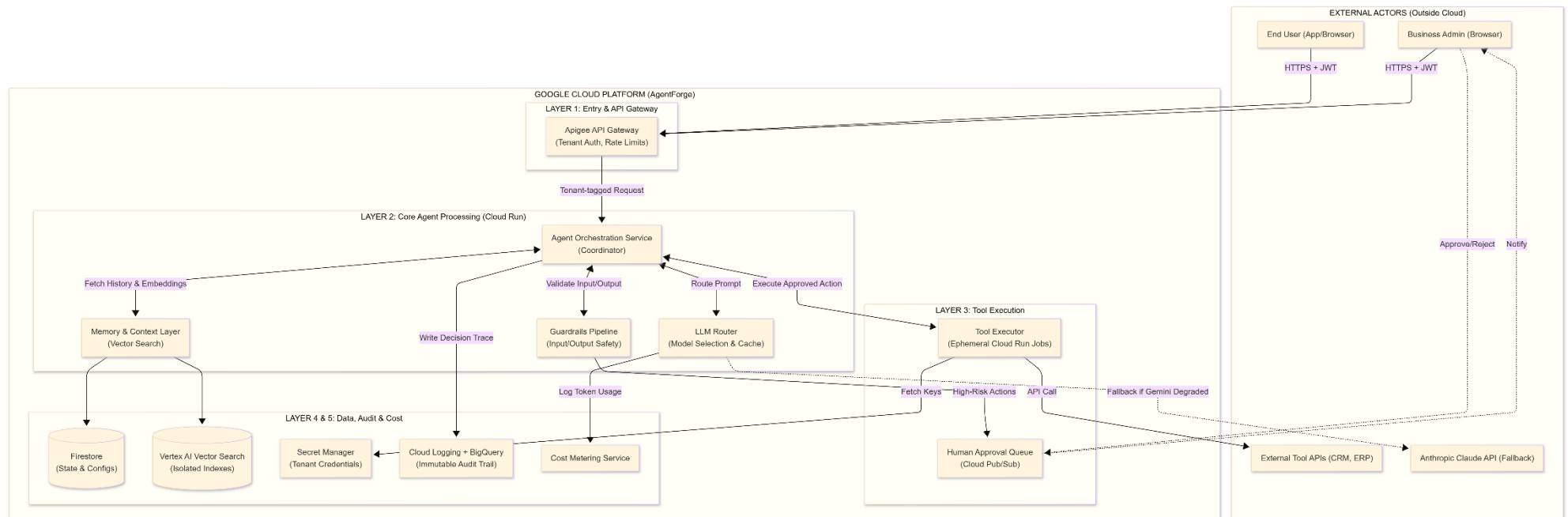
We are building **AgentForge** — a cloud platform where any business can sign up, configure an AI agent for their use case, and have it running in production within hours, without setting up a single server, database, or vector store themselves.

A business using AgentForge connects their agent to their own tools — their helpdesk system, their customer database, their order management API — through a dashboard. They define what the agent is and is not allowed to do. AgentForge then handles everything underneath: routing each conversation to the right language model, retrieving only the relevant past context rather than sending the entire history to the model, validating every action the agent proposes before it executes, logging every decision the agent makes, and stopping the agent from doing anything it shouldn't.

Here is what that looks like in practice: a customer messages a business's support bot saying "I want to cancel my subscription." AgentForge checks the conversation history, retrieves the two or three past interactions most relevant to this query, sends that context along with the message to a language model, receives the model's decision to call a "cancel subscription" tool, validates that tool call against a pre-defined safety schema, identifies it as a high-risk action, and routes it to a human reviewer before executing — all while logging every single step as a structured, queryable record. The customer receives a response in under two seconds. The agent never executes an irreversible action without authorisation. The business can query exactly what happened and why at any point in the future.

Think of AgentForge as the infrastructure layer between "a language model that can reason" and "a business system that needs to be operated safely." The business focuses on what their agent should do. We handle how it does it safely.

3. Section 3 - Architecture Diagram



4. Section 4 - Technology Choices & Justification

Cloud Run — Core Compute for All Services

We chose Cloud Run (serverless containers) for every application service in our architecture because an AI agent platform has fundamentally unpredictable traffic. A single business might be idle for hours, then spike to hundreds of concurrent agent requests when they run a product campaign. Traditional VM-based compute would either waste money idling or fail to scale fast enough. Cloud Run scales to zero when idle (zero cost) and scales to hundreds of instances within seconds during bursts.

More importantly, each service in our architecture — the Orchestration Service, Memory Layer, LLM Router, Guardrails Pipeline, and Tool Executor — scales **independently**. If we get a wave of tool-heavy requests, only the Tool Executor needs to scale; the memory layer does not. This is impossible with a monolithic deployment and unnecessarily complex with a Kubernetes cluster at our expected pilot scale. Cloud Run gives us per-service auto-scaling with no cluster management overhead.

We considered Cloud Functions and rejected it because our services hold stateful connections (Redis, Firestore) and multi-step agent reasoning can take 5–30 seconds per request — well beyond what Cloud Functions handles gracefully. We considered GKE and rejected it because it adds significant cluster management overhead without meaningful benefit at pilot scale.

Apigee — API Gateway & Multi-Tenant Entry Point

The core problem with multi-tenancy is that every request must be authenticated and attributed to the correct business tenant *before* any backend processing begins. Doing this in application code across five separate services is error-prone, duplicative, and creates opportunities for bugs that could allow one tenant to access another's resources. Apigee handles this at the perimeter: it validates the tenant's API key, injects a verified tenant context header into every downstream request, enforces per-tenant rate limits (preventing a single misbehaving tenant from consuming the entire platform's LLM quota), and meters usage per tenant for billing — all before a single line of business logic executes.

This is not just about convenience. If Apigee were not in place, a traffic spike from one tenant would be indistinguishable from platform-wide load, and we would have no reliable way to throttle or isolate it. We considered Cloud Endpoints (simpler, but lacks per-tenant policy management) and NGINX Ingress (requires manual policy management and does not integrate with GCP IAM or billing). Neither met the multi-tenant isolation requirement.

Vertex AI Vector Search — Tenant-Isolated Agent Memory

The fundamental challenge with agent memory in a multi-tenant context is cost and isolation. Sending an agent's entire conversation history to the LLM on every call is prohibitively expensive (large prompts mean high token costs and high latency) and quickly hits context window limits. We solve this using Retrieval-Augmented Generation (RAG): we embed every conversation turn and knowledge base document into vectors, store them, and at query time retrieve only the top 5 most semantically relevant chunks. The LLM only sees what is actually relevant to the current query.

We chose Vertex AI Vector Search specifically because each tenant gets a **separate vector index**. This means cross-tenant data leakage is structurally impossible — not just policy-enforced. There is no shared index that a misconfigured query could accidentally return results from. We considered Pinecone (good product, but data would reside outside GCP, complicating compliance and adding an external API dependency in a latency-critical path) and PostgreSQL with pgvector (viable at small scale, but not managed at our target throughput, and requires significant ops work as the knowledge base grows).

Gemini via Vertex AI — Primary LLM with Model Routing

We chose Gemini through Vertex AI rather than an external LLM provider for three specific reasons relevant to this problem. First, LLM inference stays within the GCP network — no external HTTPS call adds latency or creates a network boundary that could cause failures. Second, Vertex AI access is controlled by standard GCP IAM — no separate API key management for LLM calls, which removes a credential management problem. Third, all LLM requests are automatically logged through GCP Audit Logs, contributing to our audit trail without additional engineering.

We route between models based on complexity: simple, factual queries go to Gemini Flash (\$0.075/1M input tokens) and complex, multi-step reasoning goes to Gemini 1.5 Pro (\$3.50/1M input tokens). Routing 65% of traffic to Flash versus sending everything to Pro saves approximately \$110/month at pilot scale alone. If Gemini is degraded, the LLM Router automatically falls back to Claude 3.5 Sonnet via the Anthropic API. We considered OpenAI GPT-4o — excellent quality but an external API dependency with no native GCP IAM integration and higher cross-network latency.

Memorystore (Redis) — Semantic Response Cache

A significant portion of queries to a business support agent are semantically identical: "What are your business hours?" and "When do you open?" and "Are you open on weekends?" are all the same question phrased differently. Without a semantic cache, each of these triggers a separate, costly LLM call. Our semantic cache works by embedding the incoming query, checking whether any stored embedding is within a cosine similarity threshold (>0.92) of a previous response, and if so returning the cached response at near-zero cost and under 5ms latency — versus 400–800ms and real token charges for a fresh LLM call.

We estimated a 25–40% cache hit rate for a typical customer support agent, translating directly to a 25–40% reduction in LLM inference costs. This also acts as a performance buffer during traffic spikes: cache hits bypass the LLM entirely, reducing peak load on the upstream API. We considered application-layer caching and rejected it because it would not persist across Cloud Run instance restarts and would not be shared across the fleet of instances.

Firestore — Agent Configuration & Conversation State

Agent configuration data — the system prompt, allowed tools, guardrail rules, model preferences — has a flexible, nested structure that varies significantly between tenants. A relational schema would require constant migrations as we add new agent capabilities. Firestore's document model accommodates this naturally without schema changes. For conversation history, we need per-user sub-collections that can be queried efficiently by session ID — Firestore's collection-document hierarchy maps directly to this (tenants/{tenantId}/conversations/{sessionId}/messages). We considered Cloud Spanner — strong consistency is unnecessary for agent config data and adds significant cost. Cloud SQL (PostgreSQL) is viable but adds connection pool management complexity and requires schema migrations for flexible agent config.

Secret Manager — Tool Credentials per Tenant

Each business connects their agent to their own external APIs and provides API keys for those systems. These credentials are highly sensitive — a leak would allow anyone to impersonate that business's backend systems. We store each credential in Secret Manager at a path namespaced by tenant: /tenants/{tenantId}/tools/{toolName}/apiKey. The Tool Executor service account has IAM permission to read only secrets under its current tenant's namespace — cross-tenant credential access is structurally prevented, not just policy-enforced. We rejected storing credentials in Firestore (even encrypted) because application code handling decryption becomes a single point of compromise. Environment variables in Cloud Run were rejected immediately — credentials would appear in deployment configs and logs.

Cloud Logging + BigQuery — Immutable Audit Trail

The problem statement explicitly requires explaining "why an agent made a given decision — not just that it made one." This requires storing every intermediate step: what context was retrieved from memory, what prompt was sent to the LLM, what tool call the LLM requested, what the tool returned, and what the final response was. Cloud Logging captures all of this as structured JSON at sub-second granularity. A Log Sink streams these records to BigQuery, where businesses can query arbitrary questions across months of agent history using standard SQL. The BigQuery dataset has a deletion lock — the service account that writes logs has INSERT-only permissions, making the audit trail tamper-evident. We considered raw log files in Cloud Storage (queryable but requires external tooling) and Elasticsearch (powerful but operationally expensive and not GCP-native).

Cloud Pub/Sub — Human Escalation Queue & Async Decoupling

When the Guardrails Pipeline identifies a high-risk action (for example, an agent that wants to issue a refund above a configured threshold), we cannot hold the user's connection open waiting for a human reviewer — this would leave the end user staring at a loading screen indefinitely. Instead, the Guardrails Pipeline publishes the pending action to a Pub/Sub topic and immediately returns a response to the user ("This request has been sent for review"). The admin is notified separately and can approve or reject from their dashboard. Pub/Sub also decouples audit log writing from the critical response path — audit trace events are published asynchronously, so any slowdown in the audit layer does not affect response latency for the end user.

Section 5 — Scalability & Performance

Our system has four distinct load patterns that require different scaling approaches:

Pattern 1 — Gradual baseline growth: As more businesses onboard, total request volume grows proportionally. This is handled by Cloud Run's standard horizontal auto-scaling.

Pattern 2 — Per-tenant spikes: A single business runs a marketing campaign and their agent goes from 10 requests/minute to 2,000 within minutes. The critical requirement here is that this spike should not degrade any other tenant's experience at all. This is different from a typical scaling problem, where more traffic means slower responses for everyone. Our architecture prevents this through per-tenant rate limits at Apigee (which cap any single tenant before they reach backend services), per-tenant token buckets in Redis (independently metered LLM usage), per-tenant concurrency limits in the Tool Executor, and per-tenant vector search indexes (a query spike in one tenant's index has no effect on another's).

Pattern 3 — Global concurrency spikes: Multiple tenants spike simultaneously. Cloud Run scales each service independently, and Apigee begins queueing requests per-tenant before they reach backend services if any single tenant is approaching their limit.

Pattern 4 — LLM API rate limits: Scaling our platform aggressively does nothing if we hit the upstream Gemini API's tokens-per-minute ceiling. We manage this specifically: the LLM Router lowers its complexity threshold during high load, routing more traffic to Gemini Flash (which has a 10× higher rate limit than Gemini Pro). If the Gemini API approaches its rate limit, the router queues Pro-tier requests via Pub/Sub rather than returning errors. If all LLM APIs are degraded, the system returns graceful "temporarily unavailable" responses — it never silently returns stale or wrong answers.

Latency budget (typical request, no tool call, cache miss): Apigee routing: 10–20ms → embedding generation: 30–50ms → vector search retrieval: 20–40ms → guardrails input check: 15–25ms → LLM call (Gemini Flash): 400–800ms → guardrails output check: 10–20ms → Total P50: ~550ms. Audit log writing is asynchronous and does not contribute to response latency.

For chat interfaces where 800ms "thinking time" feels slow, we support streaming responses using Server-Sent Events the first tokens from the LLM appear in the user's browser within 200–300ms of the model beginning generation.

Section 6 - Security, Privacy & Compliance

The key principle across our entire security design is **structural enforcement over policy-only enforcement**: wherever possible, security is built into the structure of the infrastructure itself, so that a code bug cannot bypass it. Policy controls (IAM rules, network restrictions) add additional layers on top of structural controls.

Who can access what, and how it is enforced:

Business admins authenticate via Google OAuth 2.0 or SAML SSO. MFA is enforced for all admin accounts with no exceptions. Admin sessions are logged to Cloud Audit Logs — every configuration change, key rotation, and approval action is attributable to a specific admin identity. Each admin can only access their own tenant's data: Firestore uses path-based isolation (`/tenants/{tenantId}/...`), Vector Search uses per-tenant indexes, and Cloud Logging uses log-based access conditions that restrict tenant admins to their own `tenant_id`.

Internal services use dedicated service accounts with minimal IAM permissions. The Memory Layer service account can read Vector Search, read/write its own Firestore path, and write to Cloud Logging. It cannot access Secret Manager, call Vertex AI LLM endpoints, or read any other tenant's data. Service accounts are bound via Workload Identity Federation to specific Cloud Run service identities — credentials cannot be exported or used outside their assigned service.

End users authenticate via a business-issued API key or JWT. The business controls who gets this key. All tokens have a maximum 24-hour TTL with rotation on each use.

Encryption at rest and in transit:

All external communication (browser to Apigee, Business Admin to admin API) uses TLS 1.3. TLS 1.2 is disabled at the Apigee configuration level. All internal service-to-service communication uses Google's internal encrypted network with automatic mTLS via the Cloud Run service mesh. External tool API calls from the Tool Executor enforce TLS 1.2 minimum and reject endpoints without valid certificates.

At rest: Firestore, Vector Search indexes, Cloud Logging, and BigQuery are all encrypted with AES-256 (GCP default, Google-managed keys). Secret Manager uses Cloud KMS Customer-Managed Encryption Keys (CMEK) even Google cannot read tenant secrets without the tenant's KMS key. Memorystore (Redis) is encrypted at rest and in transit with AUTH enabled.

Consent and data minimisation:

Before sending any user message to the LLM, the Guardrails Pipeline runs a PII classifier (regex combined with a lightweight named-entity recognition model) that detects phone numbers, email addresses, credit card numbers, national ID numbers, and full names. Detected PII is replaced with a placeholder token ("My email is [EMAIL_REDACTED]") before the message reaches the LLM. The original value is stored separately in Firestore, encrypted with CMEK, retrievable only if the agent genuinely needs it (for example, to look up an order by email address). Raw user message text is never written to Cloud Logging traces contain only the scrubbed version plus a reference to the secure PII store.

Conversation histories are retained for 90 days by default, then automatically deleted via Firestore TTL policies. Knowledge base documents are indexed in Vector Search but conversation history is never embedded or stored there only business-provided documents enter the vector index.

Tool execution security:

Agents can only call tools explicitly configured by the business admin, with specific allowed URL patterns and HTTP methods. The Tool Executor validates every call against this allowlist calls to any URL not on the list are blocked unconditionally. Tool Executor instances run inside a VPC Service Controls perimeter that blocks outbound traffic to private IP ranges (10.x.x.x, 172.16.x.x, 192.168.x.x), preventing server-side request forgery attacks against internal GCP services. Tools classified as "high-risk" by the business admin require explicit human approval before execution — the agent proposes the action and a human authorises it.

Section 7 - Cost Estimate

Assumptions — Pilot Scale:

- 10 business tenants, 1 deployed agent each
- 500 agent conversations/day across all tenants = 15,000 conversations/month
- 5 turns per conversation, ~600 input tokens + ~300 output tokens per turn
- LLM routing split: 65% Gemini Flash, 30% Gemini Pro, 5% served from semantic cache
- 1.5 tool calls per conversation on average
- 5,000 knowledge base documents per tenant (~500 tokens each)

Component	Estimated Monthly Cost	Notes
LLM Inference Gemini Flash (65% of turns)	\$6.58	29.25M input + 14.63M output tokens at Flash pricing
LLM Inference Gemini Pro (30% of turns)	\$118.13	13.5M input + 6.75M output tokens at Pro pricing
Compute Cloud Run (all 6 services)	\$20.00	Scales to zero outside business hours
Redis Semantic Cache Memorystore	\$11.68	1 GB instance, 730 hours/month at \$0.016/GB-hour
Database Firestore	\$0.86	~525,000 reads + 375,000 writes/month
Vector Search Vertex AI	\$0.16	50,000 vectors stored + 75,000 queries/month
Embedding Model Vertex AI text-embedding	\$0.25	7.5M query embedding tokens + re-indexing
API Gateway Apigee	\$0.45	~150,000 API calls/month; within entry-tier pricing
Secrets Secret Manager	\$0.30	~10 tenants × \$0.03 per secret version
Async Messaging Cloud Pub/Sub	\$0.002	~37.5 MB of message data/month
Audit Logging Cloud Logging + BigQuery	\$0.00	Within GCP free tier at pilot scale
TOTAL	~\$158/month	~\$15.84 per tenant/month; ~\$0.0106 per conversation

Key cost control mechanisms:

- Semantic cache: every cache hit eliminates an LLM call, saving \$0.0017–\$0.085 per turn
- Model routing: sending 65% of traffic to Gemini Flash rather than 100% to Pro saves ~\$110/month at pilot scale
- Per-tenant token quotas: hard stop prevents any single tenant from generating runaway costs; warning alert at 80% of quota
- Conversation history pruning: only last 10 turns stored in Firestore, reducing context size and token usage per call
- Embedding reuse: knowledge base documents are embedded once at indexing time and reused across all queries

Section 8 - Availability & Resilience

Not every system requires multi-region active-active failover. For an AI agent platform, we assessed failure modes in order of severity:

- **Data loss:** not tolerable. A business's agent configuration, knowledge base, or conversation history disappearing would destroy trust and require manual reconstruction. This is our highest-priority resilience concern.
- **Silent wrong behavior:** also not tolerable. An agent that gives wrong answers or executes incorrect tool calls without anyone knowing is more dangerous than an agent that is visibly unavailable. We design every failure mode to fail visibly (return an error), never silently.
- **Brief downtime (1–5 minutes):** tolerable for this type of B2B SaaS platform. A business's customer gets an error and tries again. Unlike a safety-critical system, delayed response is not catastrophic.

How we handle specific failure scenarios:

LLM API degradation: The LLM Router monitors Gemini API error rates. If errors exceed 5% over 60 seconds, it automatically routes complex queries to Claude 3.5 Sonnet via the Anthropic API. For simple queries, it returns the closest cached response with a disclaimer. If all LLM APIs are degraded, it returns a graceful "temporarily unavailable" message. A circuit breaker pattern prevents retry storms: after 10 consecutive failures to an API, the router stops calling it for 60 seconds.

Firestore unavailability: Agent configurations are cached in Redis with a 5-minute TTL, allowing most requests to continue without Firestore. Conversation history for active sessions is cached in Redis with a 30-minute TTL. Write failures (saving new conversation turns) are queued to Pub/Sub and retried when Firestore recovers — no turn is permanently lost. Firestore uses multi-region replication within the GCP region: RPO = 0, RTO < 60 seconds for AZ-level failure.

Redis unavailability: The LLM Router detects the cache connection failure and switches to cache-bypass mode — all requests go directly to the LLM. Config reads fall back to Firestore (adds ~10ms latency, no correctness impact). Redis is deployed in high-availability mode with a replica — automatic failover within 1–2 seconds if the primary fails.

Tool call timeout: Every tool call has a hard 10-second timeout. On timeout, the Tool Executor returns a structured error to the Orchestration Service, which passes it to the LLM with a prompt: "The tool call timed out. Inform the user the information is temporarily unavailable." The agent generates a graceful, user-facing error message. The timeout is logged to the audit trail.

Agent enters a tool-calling loop: The Orchestration Service enforces a hard limit of 10 tool calls per conversation turn and 50 per session. If either limit is reached, execution stops and the agent returns a "task too complex" message. Anomaly detection alerts Platform Ops if any tenant's tool call rate exceeds three times their 7-day moving average for more than 5 minutes.

Disaster recovery: Firestore is exported daily to Cloud Storage (30-day retention). Vector Search indexes are snapshotted weekly (4-week retention). Secret Manager versions every credential change (90-day retention). At pilot scale with 10 tenants, we deploy in a single GCP region (us-central1, 3 availability zones). For a complete regional failure — extremely rare — our RTO is 4 hours (restore services from backup data in a secondary region). This is acceptable for a non-safety-critical B2B SaaS product at pilot scale. Multi-region active-active would approximately double infrastructure cost and is not justified until we have enterprise SLA customers requiring it.

Section 9 - Monitoring & Observability

For a standard web application, monitoring means: is it up? How fast is it? For an AI agent platform, those questions are necessary but not sufficient. An agent can return a 200 OK status code in 600ms with a response that is factually wrong, takes an unintended action, or violates a business rule — and standard uptime monitoring would report everything as healthy.

Our observability strategy therefore covers two distinct layers.

Infrastructure observability (is the system working as expected?):

We track per-tenant metrics in Cloud Monitoring with specific alert thresholds: request error rate (alert threshold: >5% over 5 minutes indicates systemic failure), P99 end-to-end latency (alert threshold: >3,000ms user experience degradation), LLM API error rate (alert threshold: >3% over 2 minutes triggers our fallback routing logic), tool call success rate (alert threshold: <90% over 10 minutes indicates external API degradation), guardrails block rate (alert threshold: >20% over 5 minutes possible prompt injection attack or misconfigured guardrail rule), and semantic cache hit rate (alert threshold: <10% sustained suggests cache failure or a sudden shift in query patterns).

These alerts route to PagerDuty for on-call response.

Agent behavior observability (is the agent making correct, safe decisions?):

Every agent invocation produces a **Decision Trace** a structured, ordered chain of log entries written to Cloud Logging and queryable in BigQuery. This is not a single log line; it is a linked sequence of steps that together answer "why did the agent do what it did."

For example, for a conversation where a customer asks "I want to cancel my subscription," the trace would contain: the scrubbed user message and the guardrails input check result → the top-5 memory chunks retrieved from the knowledge base with their similarity scores → the LLM call details including model used, token counts, and the tool call the model requested → the guardrails output check result showing that "cancel_subscription" was classified as high-risk and escalated to human review rather than executed → the response sent to the user → the notification sent to the admin for approval.

This trace answers questions that standard monitoring cannot: Which past context chunks influenced the agent's response? Why did the agent propose a particular action? Why was that action not immediately executed? What did this interaction cost in tokens?

Because all traces are streamed to BigQuery, business admins and Platform Ops can run arbitrary SQL queries: find all conversations from the last 30 days where the agent was blocked from executing a tool call; find which knowledge base documents are retrieved most often and might need updating; compare token usage per agent over time to identify efficiency regressions.

We additionally track agent-quality metrics per tenant in the dashboard: escalation rate (percentage of conversations routed to human review), guardrails block rate, tool call success rate, semantic cache hit rate, and average memory retrieval similarity score (a low average score suggests the knowledge base is not well-matched to the questions users are actually asking).

For diagnosing latency issues, Cloud Trace propagates a trace ID across all services — Apigee through to the Tool Executor. When P99 latency spikes, a Platform Ops engineer can select any slow request and see exactly which service was the bottleneck, without grepping through multiple separate service logs.

Section 10 - Conclusion & Future Improvements

What we built: AgentForge is a multi-tenant AI agent platform on GCP where businesses can deploy production-ready AI agents — with memory, tool-calling, safety guardrails, cost controls, and full observability — without managing any of that infrastructure themselves. Our proposal focused on three areas in depth: the 3-layer guardrails pipeline that makes tool-calling safe, the tenant-isolated RAG memory system that keeps context relevant and costs low, and the per-decision audit trail that makes agent behavior explainable.

Honest limitations:

LLM non-determinism: Our guardrails pipeline reduces the risk of harmful outputs, but it does not eliminate the possibility of a subtly wrong or misleading answer that passes all safety checks. The same input can produce different outputs on different runs. Fully addressing this requires ongoing human review of a sample of all interactions, fine-tuned models for high-stakes domains, and adversarial testing by subject-matter experts. We have the logging infrastructure for this — the review process itself is out of scope for this proposal.

Complex multi-step planning: Our architecture handles single-turn tool calls and short multi-step sequences well. It is not optimised for agents that need to execute complex, multi-session plans spanning hours or days — for example, "research this topic for 2 hours and then draft a full report." This would require a persistent task queue, long-running agent state management, and a more sophisticated planning architecture. It is an extension of the current design, not an incompatibility, but it is not in this proposal.

Knowledge base freshness: Our vector index is re-indexed incrementally on a weekly schedule. For businesses with rapidly changing information (daily price changes, new product launches), weekly re-indexing is too slow. Real-time streaming updates to the vector index would require a streaming indexing pipeline, which adds cost and complexity we chose not to design in detail here.

Configuration dashboard UX: The backend infrastructure is described in depth, but the business-facing dashboard for configuring agents, connecting tools, and reviewing audit logs is mentioned but not designed. Poor UX on the configuration side would limit adoption regardless of how capable the backend is.

What we would build next (priority order):

Priority 1 — Multi-turn planning with persistent task state: Extend the Orchestration Service to support multi-step plans with checkpointed state in Firestore, allowing agents to break complex requests into sub-tasks, execute them sequentially or in parallel, and report progress back to the user. Add streaming response delivery so the first tokens appear in the browser within 200ms rather than the user waiting for the full response.

Priority 2 — Agent template marketplace: Allow businesses to share anonymised agent configurations with other AgentForge customers. A "Customer Support Agent" template pre-configured with common helpdesk tool integrations, starter guardrail rules, and a sample knowledge base could reduce onboarding time from hours to minutes and create a platform network effect.

Priority 3 — Fine-tuning pipeline for high-volume tenants: For tenants where Gemini Flash's general-purpose quality is insufficient for their specific domain, provide a pipeline to fine-tune a smaller base model on their conversation history and knowledge base. A well-tuned smaller model can outperform a general-purpose large model on specific tasks at 10–20× lower cost per token.

Priority 4 — Real-time knowledge base updates: Build a streaming indexing pipeline (document change → Cloud Pub/Sub → Embedding Service → Vector Search upsert) that updates the vector index within seconds of a knowledge base document being modified. This enables agents for fast-moving business domains.

Priority 5 — Multi-region active-active deployment: As the platform grows to serve enterprise customers with strict SLA requirements, deploy the full stack in two GCP regions (us-central1 + europe-west1) with active-active traffic routing and Firestore multi-region replication. This reduces RTO for a complete regional failure from the current 4 hours to under 1 minute.

The most important design decision in this entire proposal is treating safety and auditability as foundational — not features added after the core system was built. Every tool call is validated before it executes. Every agent decision is traced. Every guardrail block is logged with full context. A business that deploys an AI agent that can take real actions in their systems must be able to trust it. That trust is built through transparency and control, not through capability alone.